

✓ Mini GPT from Scratch

A tiny Transformer that learns to predict the next character.

We'll build every piece by hand:

- Token & Positional Embeddings
- Self-Attention (Q, K, V)
- Multi-Head Attention
- Feed-Forward Network
- Residual Connections + LayerNorm
- Full GPT Model
- Training Loop
- Text Generation

By the end, our model will learn patterns from a tiny dataset and generate new sentences. 🚀

✓ 1. Setup

Import PyTorch and set a seed so results are reproducible.

```
import torch
import torch.nn as nn
import torch.nn.functional as F

torch.manual_seed(42)
device = 'cuda' if torch.cuda.is_available() else 'cpu'
print(f"Using device: {device}")
```

Using device: cuda

✓ 2. Training Data

A tiny dataset of simple sentences. The model will learn the patterns:

- "the cat sat on the ____"
- "the dog ____ in the ____"
- etc.

```
text = """the cat sat on the mat
the dog sat on the floor
the bird sat on the branch
```

```

the cat sat on the chair
the dog ran in the park
the cat slept on the mat
the bird flew in the sky
the cat played with the ball
the dog ate the bone
the cat drank the milk
.....

```

```

print(f"Total characters: {len(text)}")
print("Sample:")
print(text[:80])

```

```

Total characters: 247
Sample:
the cat sat on the mat
the dog sat on the floor
the bird sat on the branch
the c

```

✓ 3. Tokenizer (Character-Level)

Every unique character → an integer ID.

This is the simplest possible tokenizer. Real LLMs use BPE (subword) tokens, but characters are perfect for learning.

```

chars = sorted(list(set(text)))
vocab_size = len(chars)
stoi = {c: i for i, c in enumerate(chars)}
itos = {i: c for i, c in enumerate(chars)}

encode = lambda s: [stoi[c] for c in s]
decode = lambda ids: ''.join(itos[i] for i in ids)

print(f"Vocab size: {vocab_size}")
print(f"Characters: {chars}")
print(f"\nEncode 'cat': {encode('cat')}")
print(f"Decode back: {decode(encode('cat'))}")

```

```

Vocab size: 22
Characters: ['\n', ' ', 'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', '
', 'l', 'o', 'n', 'p', 'r', 's', 't']

Encode 'cat': [4, 2, 19]
Decode back: cat

```

[link text](#)## 4. Hyperparameters

Tiny values so it trains in seconds on CPU.

```

n_embd      = 64      # vector size per token
n_head      = 4        # number of attention heads
n_layer     = 2        # number of transformer blocks
block_size  = 16       # context window
batch_size  = 16
max_iters   = 2000
lr          = 3e-3
dropout     = 0.1

```

```

data = torch.tensor(encode(text), dtype=torch.long)
print(f"Data tensor shape: {data.shape}")
print(f"First 30 tokens: {data[:30].tolist()}")

```

```

Data tensor shape: torch.Size([247])
First 30 tokens: [19, 9, 6, 1, 4, 2, 19, 1, 18, 2, 19, 1, 15, 14, 1, 1

```

```
# This is formatted as code
```

✓ 5. Batch Sampler

Each training example is `(input, target)` where target = input shifted by 1.

So if input is `"the cat"`, target is `"he cat "` — the model learns to predict the **next character** at every position.

```

def get_batch():
    ix = torch.randint(0, len(data) - block_size - 1, (1,))
    x = torch.stack([data[i:i+block_size] for i in ix])
    y = torch.stack([data[i+1:i+block_size+1] for i in ix])
    return x.to(device), y.to(device)

```

```

# Peek at one batch
xb, yb = get_batch()
print(f"Input shape: {xb.shape}")
print(f"Target shape: {yb.shape}")
print(f"\nInput sample: '{decode(xb[0].tolist())}'")
print(f"Target sample: '{decode(yb[0].tolist())}'")

```

```

Input shape: torch.Size([16, 16])
Target shape: torch.Size([16, 16])

```

```

Input sample: 'n the branch
the'
Target sample: ' the branch
the '

```

✓ 6. Single Attention Head

The heart of the transformer. Each token produces three vectors:

- **Q (Query)** — *"what am I looking for?"*
- **K (Key)** — *"what do I offer?"*
- **V (Value)** — *"what content do I carry?"*

Then: $\text{softmax}(Q \cdot K^T / \sqrt{d}) \cdot V \rightarrow$ each token gets a context-aware version of itself.

The mask makes it **causal** — tokens can only see the past, not the future.

```
class Head(nn.Module):
    def __init__(self, head_size):
        super().__init__()
        self.Wq = nn.Linear(n_embd, head_size, bias=False)
        self.Wk = nn.Linear(n_embd, head_size, bias=False)
        self.Wv = nn.Linear(n_embd, head_size, bias=False)
        self.register_buffer('mask', torch.tril(torch.ones(block_size, block_size)))
        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        B, T, C = x.shape
        q = self.Wq(x)
        k = self.Wk(x)
        v = self.Wv(x)

        # Q · K^T → similarity scores
        scores = q @ k.transpose(-2, -1) / (k.shape[-1] ** 0.5)
        # Mask future tokens
        scores = scores.masked_fill(self.mask[:T, :T] == 0, float('-i'))
        # Softmax → attention weights
        weights = F.softmax(scores, dim=-1)
        weights = self.dropout(weights)

        # Mix values using weights
        return weights @ v

print('✓ Head class defined')
```

✓ Head class defined

✓ 7. Multi-Head Attention

Run multiple attention heads in **parallel**, each focusing on different patterns.

Then concatenate their outputs and project back to n_embd dims.

```
class MultiHeadAttention(nn.Module):
    def __init__(self, n_head):
        super().__init__()
        head_size = n_embd // n_head
        self.heads = nn.ModuleList([Head(head_size) for _ in range(n_head)])
        self.proj = nn.Linear(n_embd, n_embd)
```

```

self.dropout = nn.Dropout(dropout)

def forward(self, x):
    out = torch.cat([h(x) for h in self.heads], dim=-1)
    return self.dropout(self.proj(out))

print('✓ MultiHeadAttention class defined')

```

✓ MultiHeadAttention class defined

✓ 8. Feed-Forward Network

After attention gathered context, FFN does **per-token thinking**:

1. **Expand** ($n_embd \rightarrow 4 \times n_embd$) — open up "scratch space"
2. **GELU** — non-linear filter, keeps strong features
3. **Squash** ($4 \times n_embd \rightarrow n_embd$) — write knowledge back

This is where most of the model's **knowledge** lives.

```

class FeedForward(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(n_embd, 4 * n_embd),  # W1: expand
            nn.GELU(),                       # filter
            nn.Linear(4 * n_embd, n_embd),   # W2: squash
            nn.Dropout(dropout),
        )

    def forward(self, x):
        return self.net(x)

print('✓ FeedForward class defined')

```

✓ FeedForward class defined

✓ 9. Transformer Block

One full block = LayerNorm → Attention → Residual → LayerNorm → FFN → Residual.

- **LayerNorm** keeps numbers stable across deep stacks.
- **Residual** ($x + \dots$) preserves original info and lets gradients flow back.

```

class TransformerBlock(nn.Module):
    def __init__(self):
        super().__init__()
        self.ln1 = nn.LayerNorm(n_embd)

```

```

self.attn = MultiHeadAttention(n_head)
self.ln2 = nn.LayerNorm(n_embd)
self.ffn = FeedForward()

def forward(self, x):
    x = x + self.attn(self.ln1(x))    # attention + residual
    x = x + self.ffn(self.ln2(x))    # FFN + residual
    return x

print('✓ TransformerBlock class defined')

```

✓ TransformerBlock class defined

✓ 10. The Full Mini GPT

Stack everything together:

tokens → embed + position → [Block × N] → LayerNorm → Head → next-token p

The **Head** (final linear layer) projects each token vector onto the vocabulary.
Highest-scoring word = predicted next token.

```

class MiniGPT(nn.Module):
    def __init__(self):
        super().__init__()
        self.token_embedding = nn.Embedding(vocab_size, n_embd)
        self.pos_embedding = nn.Embedding(block_size, n_embd)
        self.blocks = nn.Sequential(*[TransformerBlock() for _ in range(N)])
        self.ln_f = nn.LayerNorm(n_embd)
        self.head = nn.Linear(n_embd, vocab_size, bias=False)

    def forward(self, idx, targets=None):
        B, T = idx.shape
        tok = self.token_embedding(idx)
        pos = self.pos_embedding(torch.arange(T, device=idx.device))
        x = tok + pos

        x = self.blocks(x)
        x = self.ln_f(x)
        logits = self.head(x)

        loss = None
        if targets is not None:
            loss = F.cross_entropy(logits.view(-1, vocab_size), targets.view(-1))
        return logits, loss

    @torch.no_grad()
    def generate(self, idx, max_new_tokens):
        for _ in range(max_new_tokens):
            idx_cond = idx[:, -block_size:]
            logits, _ = self(idx_cond)

```

```

logits = logits[:, -1, :]
probs = F.softmax(logits, dim=-1)
next_id = torch.multinomial(probs, num_samples=1)
idx = torch.cat((idx, next_id), dim=1)
return idx

model = MiniGPT().to(device)
n_params = sum(p.numel() for p in model.parameters())
print(f'✓ Model built! Total parameters: {n_params:,}')
```

✓ Model built! Total parameters: 103,552 (~103.6K)

✓ 11. Generation BEFORE Training

Let's see what the model produces with **random weights**. It should be gibberish — pure noise.

This is the "before" picture. 🤖

```

prompt = "the cat sat on "
context = torch.tensor([encode(prompt)], dtype=torch.long, device=device)
generated = model.generate(context, max_new_tokens=80)

print("BEFORE TRAINING (random weights):")
print("-" * 50)
print(decode(generated[0].tolist()))
print("-" * 50)
print("\n^ Pure gibberish – model knows nothing yet.")
```

```

BEFORE TRAINING (random weights):
-----
the cat sat on nlifsltmbghoeorccclfpwbcfcbdsfpossotoeslsadrko
kdcekrsmfyfdtnogfsg
fcfcoootcmomis
-----
```

^ Pure gibberish – model knows nothing yet.

✓ 12. Training Loop

Now we train. Watch the **loss drop** from ~3.0 → ~0.2.

Each step:

1. Sample a batch
2. Forward pass → compute loss
3. Backprop → compute gradients
4. Optimizer step → update **all** weights (embeddings, $W_q/W_k/W_v$, W_1/W_2 , vocab vectors)

```
optimizer = torch.optim.AdamW(model.parameters(), lr=lr)

print("--- Training ---")
losses = []
for step in range(max_iters):
    x, y = get_batch()
    logits, loss = model(x, y)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    losses.append(loss.item())

    if step % 200 == 0:
        print(f"step {step:4d} | loss {loss.item():.4f}")

print(f"\nFinal loss: {loss.item():.4f}")
print(f"Loss dropped from {losses[0]:.2f} → {losses[-1]:.2f}")
```

```
--- Training ---
step    0 | loss 3.3375
step  200 | loss 0.2822
step  400 | loss 0.2615
step  600 | loss 0.2623
step  800 | loss 0.2409
step 1000 | loss 0.2069
step 1200 | loss 0.1846
step 1400 | loss 0.2435
step 1600 | loss 0.1846
step 1800 | loss 0.2139

Final loss: 0.2207
Loss dropped from 3.34 → 0.22 ✓
```

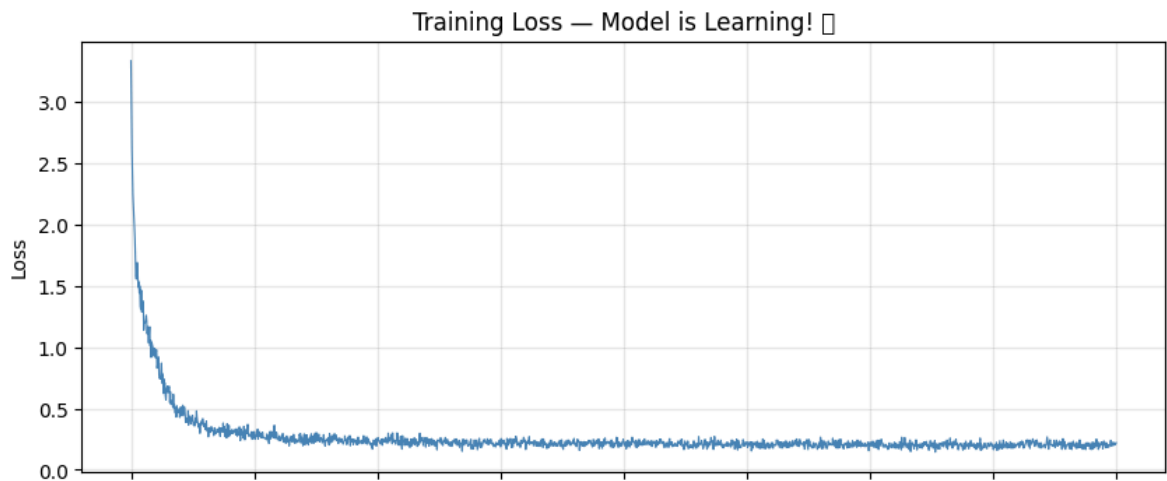
✓ 13. Visualize the Loss Curve

A picture is worth a thousand prints — let's see the model learning.

```
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 4))
plt.plot(losses, color='steelblue', linewidth=0.8)
plt.title('Training Loss – Model is Learning! 📈')
plt.xlabel('Step')
plt.ylabel('Loss')
plt.grid(alpha=0.3)
plt.show()
```

```
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:151
fig.canvas.print_figure(bytes_io, **kw)
```



✓ 14. Generation AFTER Training

Now let's see what our model has learned. It should produce **real sentences** matching the patterns it saw. ✨

```
prompt = "the cat sat on "
context = torch.tensor([encode(prompt)], dtype=torch.long, device=device)
generated = model.generate(context, max_new_tokens=120)

print("AFTER TRAINING:")
print("=" * 50)
print(decode(generated[0].tolist()))
print("=" * 50)
```

AFTER TRAINING:

```
=====
the cat sat on the floor
the bird sat on the branch
the cat sat on the park
the cat slept on the mat
the bird fleyd with the ball
the d
=====
```

✓ 15. Try Different Prompts

The model has learned the structure of our tiny world. Try several prompts and see what it generates.

```
prompts = [
    "the cat ",
    "the dog ",
    "the bird ",
    "the cat sat on ",
```

```

    "the dog ran ",
]

for p in prompts:
    ctx = torch.tensor([encode(p)], dtype=torch.long, device=device)
    out = model.generate(ctx, max_new_tokens=40)
    print(f"'{p}' → '{decode(out[0].tolist())}'")
    print()

```

```

'the cat ' → 'the cat drank the milky
the dog sat played with '

```

```

'the dog ' → 'the dog ate the bone
the cat drank the milky
the'

```

```

'the bird ' → 'the bird sat on the flone
the cat drank the milk
'

```

```

'the cat sat on ' → 'the cat sat on the floor
the bird sat on the branch
the'

```

```

'the dog ran ' → 'the dog ran in the park
the cat slept on the mat
the'

```

16. Peek Inside — Attention Weights

Let's see what the model is attending to. For each character, which previous characters does it look at?

Brighter = more attention. Diagonal/lower-triangular pattern is expected (causal mask).

✓ What We Built

Component	Job
Tokenizer	Words → IDs
Embedding	IDs → meaning vectors
Position	Add order info
Attention	"Who do I relate to?"
FFN	"What's known about me?"
Residual + LayerNorm	Keep deep stack stable
Head	Pick next word

Key Takeaway

Build context-rich vector → match to closest vocab word. 🚀

That's the entire transformer.

Next Steps for Students

- Increase `n_layer`, `n_embd` for bigger model
- Train on real data (Shakespeare, your code, your notes)
- Add temperature/top-k to generation
- Try fine-tuning with **LoRA** on `Wq`, `Wk`, `Wv`!

```
# =====
# FINE-TUNING DEMO (fixed – only uses existing vocab chars)
# =====

# Check what chars are available
print(f"Available chars: {chars}")

# Step 1: New domain data – using ONLY chars from original vocab
# (no new letters introduced)
new_text = """the cat sat on the perch
the cat sat on the tower
the cat sat on the perch
the cat sat on the tower
the cat sat on the perch
the cat sat on the tower
the cat napped on the perch
the cat napped on the tower
the cat sat on the perch
the cat sat on the tower
the cat sat on the perch
the cat sat on the tower
"""

# Verify no new chars
new_chars = set(new_text) - set(chars)
print(f"New chars introduced: {new_chars} (should be empty)")

# Step 2: Convert to tensor
new_data = torch.tensor(encode(new_text), dtype=torch.long)
print(f"New data length: {len(new_data)}")

def get_finetune_batch():
    ix = torch.randint(0, len(new_data) - block_size - 1, (batch_size,))
    x = torch.stack([new_data[i:i+block_size] for i in ix])
    y = torch.stack([new_data[i+1:i+block_size+1] for i in ix])
    return x.to(device), y.to(device)

# Step 3: BEFORE fine-tuning
print("\n--- BEFORE Fine-tuning ---")
prompt = "the cat sat on "
ctx = torch.tensor([encode(prompt)], dtype=torch.long, device=device)
```

```

out = model.generate(ctx, max_new_tokens=40)
print(f'''{decode(out[0].tolist())}''')

# Step 4: Fine-tune
finetune_optimizer = torch.optim.AdamW(model.parameters(), lr=1e-4)
finetune_steps = 500

print("\n--- Fine-tuning ---")
for step in range(finete_steps):
    x, y = get_finetune_batch()
    logits, loss = model(x, y)
    finetune_optimizer.zero_grad()
    loss.backward()
    finetune_optimizer.step()
    if step % 100 == 0:
        print(f"step {step:3d} | loss {loss.item():.4f}")

# Step 5: AFTER fine-tuning
print("\n--- AFTER Fine-tuning ---")
for p in ["the cat sat on ", "the cat napped on "]:
    ctx = torch.tensor([encode(p)], dtype=torch.long, device=device)
    out = model.generate(ctx, max_new_tokens=30)
    print(f'''{p}' → '{decode(out[0].tolist())}''')

# Step 6: Forgetting check
print("\n--- Catastrophic Forgetting Check (old prompts) ---")
for p in ["the dog ran ", "the bird flew "]:
    ctx = torch.tensor([encode(p)], dtype=torch.long, device=device)
    out = model.generate(ctx, max_new_tokens=30)
    print(f'''{p}' → '{decode(out[0].tolist())}''')

```

Available chars: ['\n', ' ', 'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i']
 New chars introduced: set() (should be empty)

New data length: 306

--- BEFORE Fine-tuning ---

'the cat sat on the chair
 the dog ran in the park
 the ca'

--- Fine-tuning ---

step 0 | loss 2.3904
 step 100 | loss 0.6986
 step 200 | loss 0.5857
 step 300 | loss 0.3934
 step 400 | loss 0.2849

--- AFTER Fine-tuning ---

'the cat sat on ' → 'the cat sat on the perch
 the cat sat on the t'
 'the cat napped on ' → 'the cat napped on the perch
 the cat sat on the t'

--- Catastrophic Forgetting Check (old prompts) ---

'the dog ran ' → 'the dog ran in played with tower
 the cat s'

```
'the bird flew ' → 'the bird flew in the sat on the perch  
the ca'
```